Chapter 5: For Loops

Repetition Structure: Defined-Count Loops

Course 040223101 – Computer Programming for Mathematics 1

Department of Mathematics, Faculty of Applied Science, King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul | Semester 1, Academic Year 2569

Chapter Learning Objectives

Upon completing this chapter, students will be able to:

01

Explain the for Loop

Describe the working principle of defined-count repetition using the `for` statement.

02

Use range()

Apply the `range()` function to generate numeric sequences for iteration.

03

Compute Sums, Products & Series

Write programs to calculate summations, products, and mathematical series using `for` loops.

04

Nested Loops

Use nested loops to solve two-dimensional problems such as multiplication tables.

05

Apply to Mathematics

Apply `for` loops to solve mathematical problems. (Aligned with CLO 3)

Why Do We Need Loops?

Many tasks require executing the same instruction repeatedly. For example, finding the sum of numbers 1 to 100 – writing each addition on a separate line would be impractical. The **repetition structure (loop)** allows us to issue repeated commands concisely.

This chapter focuses on the **for loop**, which is ideal when the **exact number of repetitions is known in advance**. It is one of the most fundamental tools in mathematical programming.

5.1 The `for` Statement and `range()`

The `for` statement iterates over members of a sequence. It is most commonly paired with the `range()` function, which generates a sequence of integers. The loop variable (e.g., `i`) receives each value from the sequence one at a time per iteration.

`range(n)`

Generates integers from **0 to n−1**.

Example: `range(5)` → 0, 1, 2, 3, 4

`range(a, b)`

Generates integers from **a to b−1**.

Example: `range(2, 6)` → 2, 3, 4, 5

`range(a, b, s)`

Generates integers from **a to b−1**, stepping by **s**.

Example: `range(1, 10, 2)` → 1, 3, 5, 7, 9

range() – Key Rules to Remember

Important Rule

`range(n)` starts at **0** and ends at **n−1**. The value **n itself is NOT included**.

Therefore, `range(1, 6)` produces: 1, 2, 3, 4, 5 – exactly five numbers.

range() Summary Table

Form	Meaning	Sequence
`range(n)`	0 to n−1	`range(5)` → 0,1,2,3,4
`range(a, b)`	a to b−1	`range(2,6)` → 2,3,4,5
`range(a, b, s)`	a to b−1, step s	`range(1,10,2)` → 1,3,5,7,9

First for Loop Example – File: 05for1.py

The simplest use of a `for` loop: printing a message for each iteration. The variable `i` takes each value from `range(5)` in turn.

Source Code

```
for i in range(5):
    print("รอบที่", i)
```

Output

```
รอบที่ 0
รอบที่ 1
รอบที่ 2
รอบที่ 3
รอบที่ 4
```

The loop runs exactly 5 times. Notice that the count starts at 0, not 1. This is the default behavior of `range(n)`.

5.2 Accumulating a Sum with for

One of the most frequently used techniques is **accumulating a value in a variable**. The pattern is: initialize an accumulator variable before the loop, then add to it in each iteration. This is the foundation for computing sums, products, and series.

Step 1: Initialize

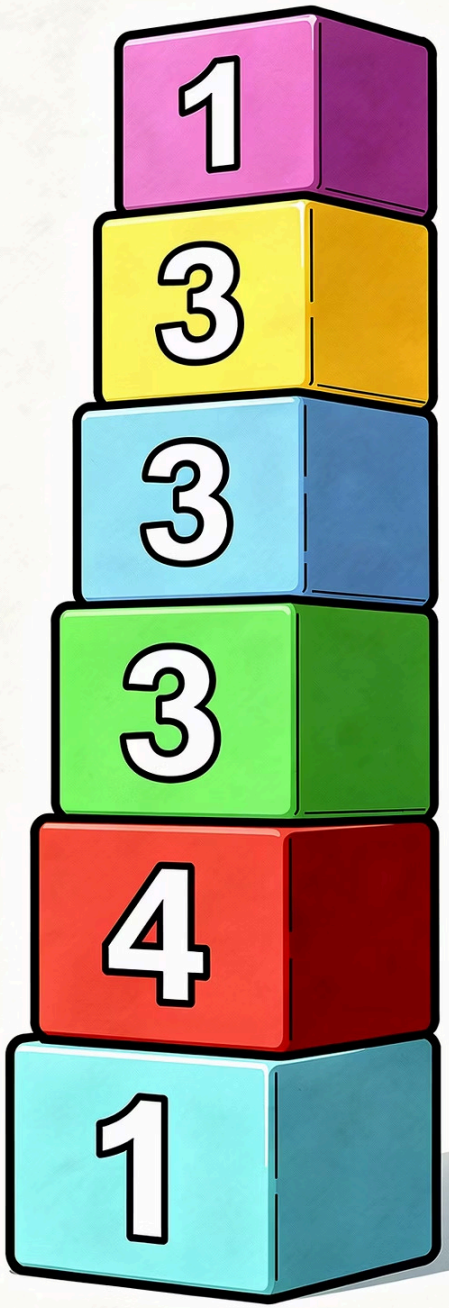
Set `total = 0` before entering the loop. The accumulator must start at zero for summation.

Step 2: Accumulate

Inside the loop, write `total = total + i` (or the shorthand `total += i`) to add each value.

Step 3: Output

After the loop ends, print the final accumulated value.



Sum of 1 to 100 – File: 05sum.py

This classic example computes the sum of all integers from 1 to 100 using the accumulator pattern. The result matches the well-known formula $\frac{n(n+1)}{2} = \frac{100 \times 101}{2} = 5050$.

Source Code

```
total = 0
for i in range(1, 101):
    total = total + i
print("ผลรวม 1 ถึง 100 =", total)
```

Output

```
ผลรวม 1 ถึง 100 = 5050
```

Shorthand Notation

The statement `total = total + i` can be written more concisely as `total += i`. Both produce identical results.

Summation Formula Verification

The program result of **5050** can be independently verified using the closed-form formula for the sum of the first n natural numbers:

$$S = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{100 \times 101}{2} = 5050$$

This confirms that the loop-based accumulation approach is mathematically correct. The `for` loop essentially computes the discrete summation operator $\sum$ step by step.

> ✓ Programming and mathematics align perfectly here – the loop variable `i` directly corresponds to the summation index in the formula.

5.3 Computing Products and Factorials

For multiplicative accumulation, the initial value of the accumulator must be **1**, not 0. This is because multiplying by 0 always yields 0, which would destroy the product. The factorial function is the classic example:

$$n! = 1 \times 2 \times 3 \times \cdots \times n$$

Sum Accumulator

Initialize to `0`

Use `+=` operator

Identity element for addition

Product Accumulator

Initialize to `1`

Use `*=` operator

Identity element for multiplication

Factorial Program – File: 05factorial.py

The program reads an integer `n` from the user and computes $n!$ by multiplying all integers from 1 to n using a `for` loop.

Source Code

```
n = int(input("ใส่ n: "))
fact = 1
for i in range(1, n + 1):
    fact = fact * i
print(f"{n}! = {fact}")
```

Output

```
ใส่ n: 5
5! = 120
```

Verification

$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$

Note: `range(1, n+1)` ensures the loop includes `n` itself, since `range` excludes the upper bound.

Factorial Values at a Glance

1

0! and 1!

Both equal 1 by definition

120

5!

Program output for n = 5

720

6!

1×2×3×4×5×6

5040

7!

Factorials grow very rapidly

Factorials grow extremely fast – this is why the loop-based approach is essential. Writing out each multiplication manually becomes infeasible for large `n`.

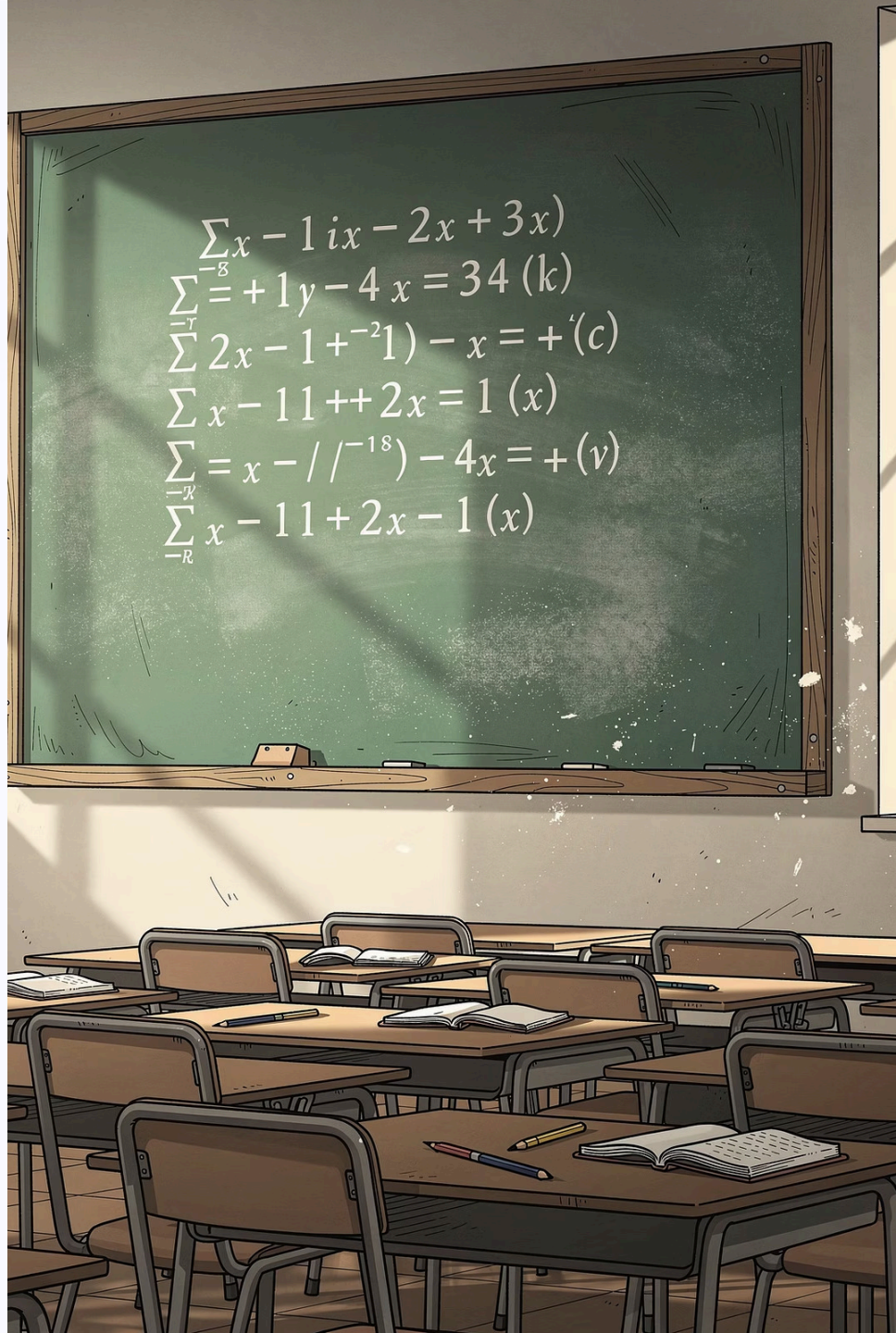
5.4 Computing Mathematical Series

The `for` loop is well-suited for computing mathematical series – sums of sequences defined by a formula. The general approach is the same accumulator pattern, but the value added each iteration follows a mathematical rule.

A classic example is the **harmonic series**:

$$S = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{i=1}^{n} \frac{1}{i}$$

Each term $\frac{1}{i}$ is computed inside the loop and added to the running sum `s`.



Harmonic Series Program – File: 05series.py

Source Code

```
n = int(input("ใส่ n: "))
s = 0
for i in range(1, n + 1):
    s += 1 / i
print(f"ผลรวมอนุกรม = {s:.4f}")
```

Output (n = 5)

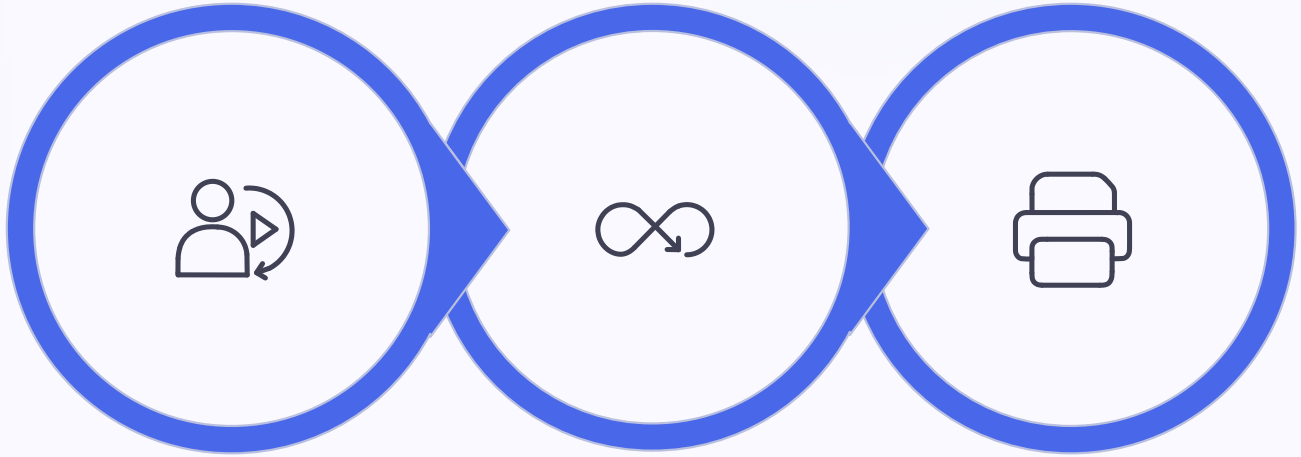
```
ใส่ n: 5
ผลรวมอนุกรม = 2.2833
```

Manual Verification

$$\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} = 1 + 0.5 + 0.333 + 0.25 + 0.2 = 2.2833$$

The format specifier `:.4f` in the f-string rounds the output to 4 decimal places, which is standard for displaying series approximations.

Series Accumulation Pattern



Initialize Accumulate Output

This three-step pattern – initialize, accumulate in loop, output – applies to virtually all series computations in mathematics. Mastering it unlocks the ability to approximate integrals, compute probabilities, and evaluate infinite sums numerically.

5.5 Nested Loops – Loops Inside Loops

A **nested loop** is a loop placed inside another loop. The **inner loop completes all its iterations** every time the outer loop executes one iteration. This structure is essential for solving two-dimensional problems.

Outer Loop

Controls the "row" dimension. Runs once per outer iteration. In a multiplication table, this is the multiplier (e.g., 1, 2, 3).

Inner Loop

Controls the "column" dimension. Runs **completely** for each outer iteration. In a multiplication table, this is the multiplicand (e.g., 1 through 5).

Total Iterations

If the outer loop runs *m* times and the inner loop runs *n* times, the inner body executes **m × n** times total.

Multiplication Table – File: 05nested.py

This example uses a nested loop to print a partial multiplication table. The outer loop iterates over multipliers 1 - 3; the inner loop iterates over multiplicands 1 - 5.

Source Code

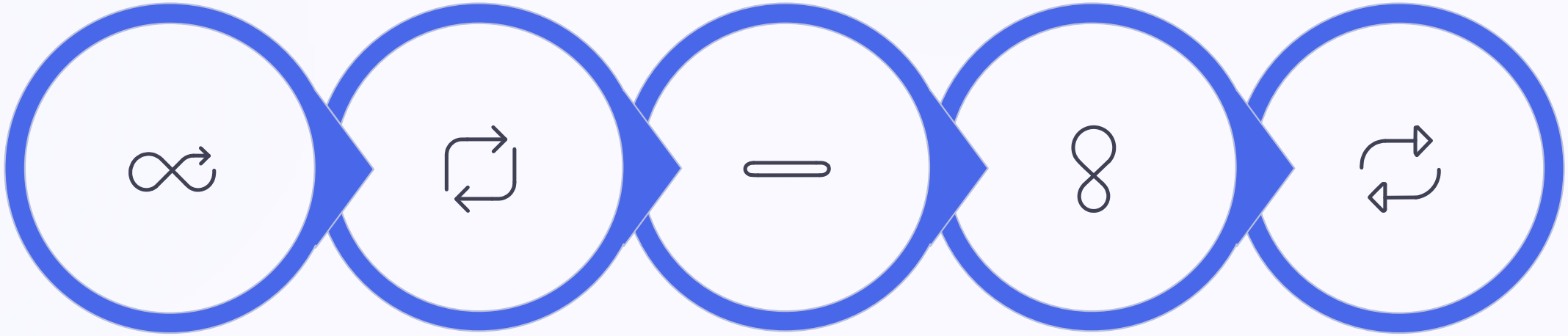
```
for i in range(1, 4):
    for j in range(1, 6):
        print(f"{i} x {j} = {i * j}")
    print("-----")
```

Output (partial)

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
-----
2 x 1 = 2
...
3 x 5 = 15
```

The `print("-----")` statement is at the outer loop's indentation level, so it prints once after each complete inner loop cycle – creating a visual separator between rows.

How Nested Loops Execute



Understanding the execution order of nested loops is critical. The indentation in Python determines which statements belong to which loop. Code at the inner loop's indentation runs **m × n** times; code at the outer loop's indentation runs only **m** times.

5.6 Applied Example: Prime Number Check

A **prime number** is an integer greater than 1 that is divisible only by 1 and itself. We can use a `for` loop to test divisibility by all integers from 2 to n−1. If any divisor is found, the number is not prime.

Initialize Flag

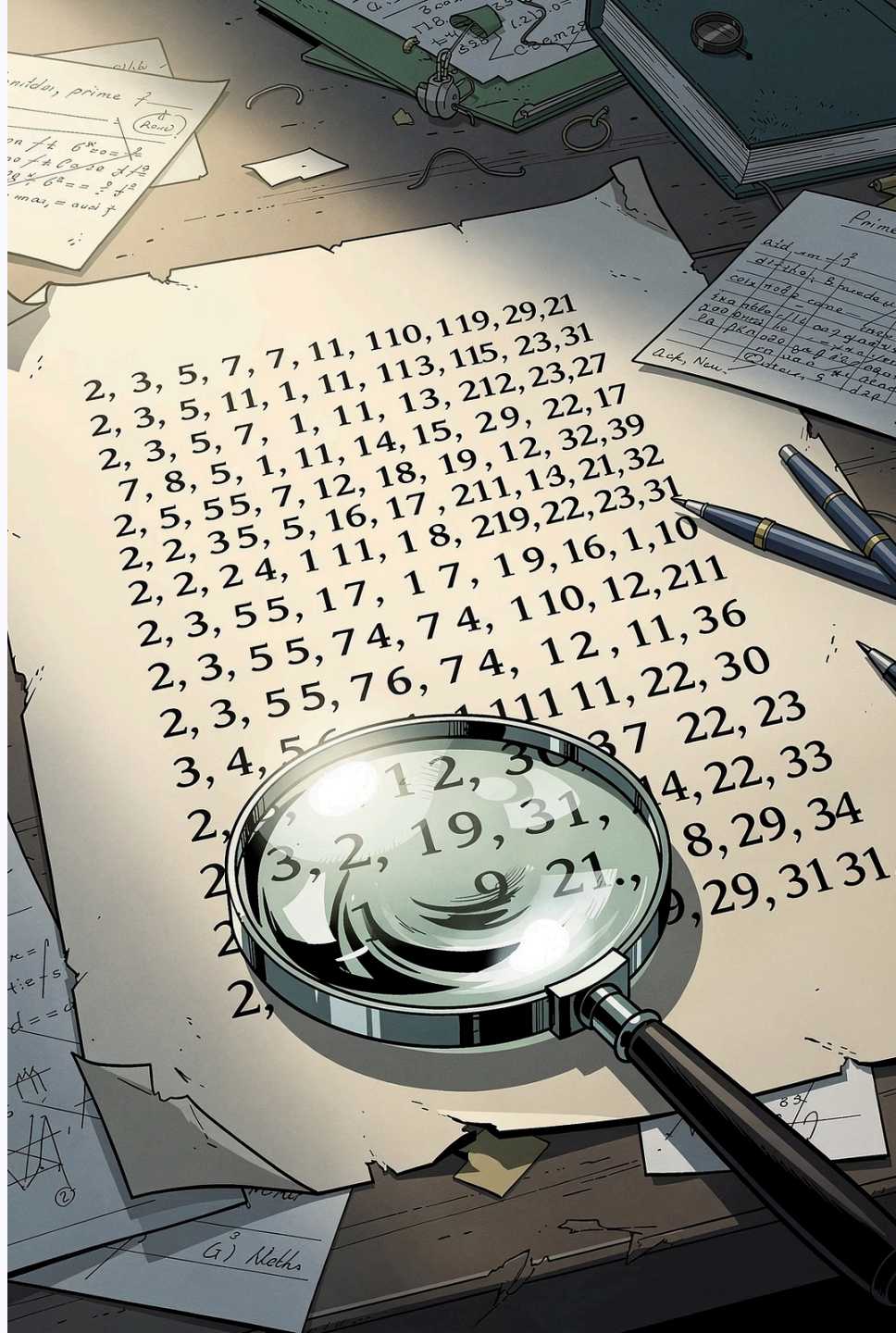
Set `is_prime = True` before the loop. Also handle the special case: if `n < 2`, immediately set `is_prime = False`.

Test Each Divisor

Loop `i` from 2 to n−1. If `n % i == 0` (i.e., `i` divides `n` evenly), set `is_prime = False`.

Report Result

After the loop, check the flag and print whether `n` is prime or not.



Prime Number Program – File: 05prime.py

Source Code

```
n = int(input("ใส่จำนวนเต็ม: "))
is_prime = True
if n < 2:
    is_prime = False
for i in range(2, n):
    if n % i == 0:
        is_prime = False
if is_prime:
    print(f"{n} เป็นจำนวนเฉพาะ")
else:
    print(f"{n} ไม่เป็นจำนวนเฉพาะ")
```

Output

```
ใส่จำนวนเต็ม: 13
13 เป็นจำนวนเฉพาะ
```

Why it works

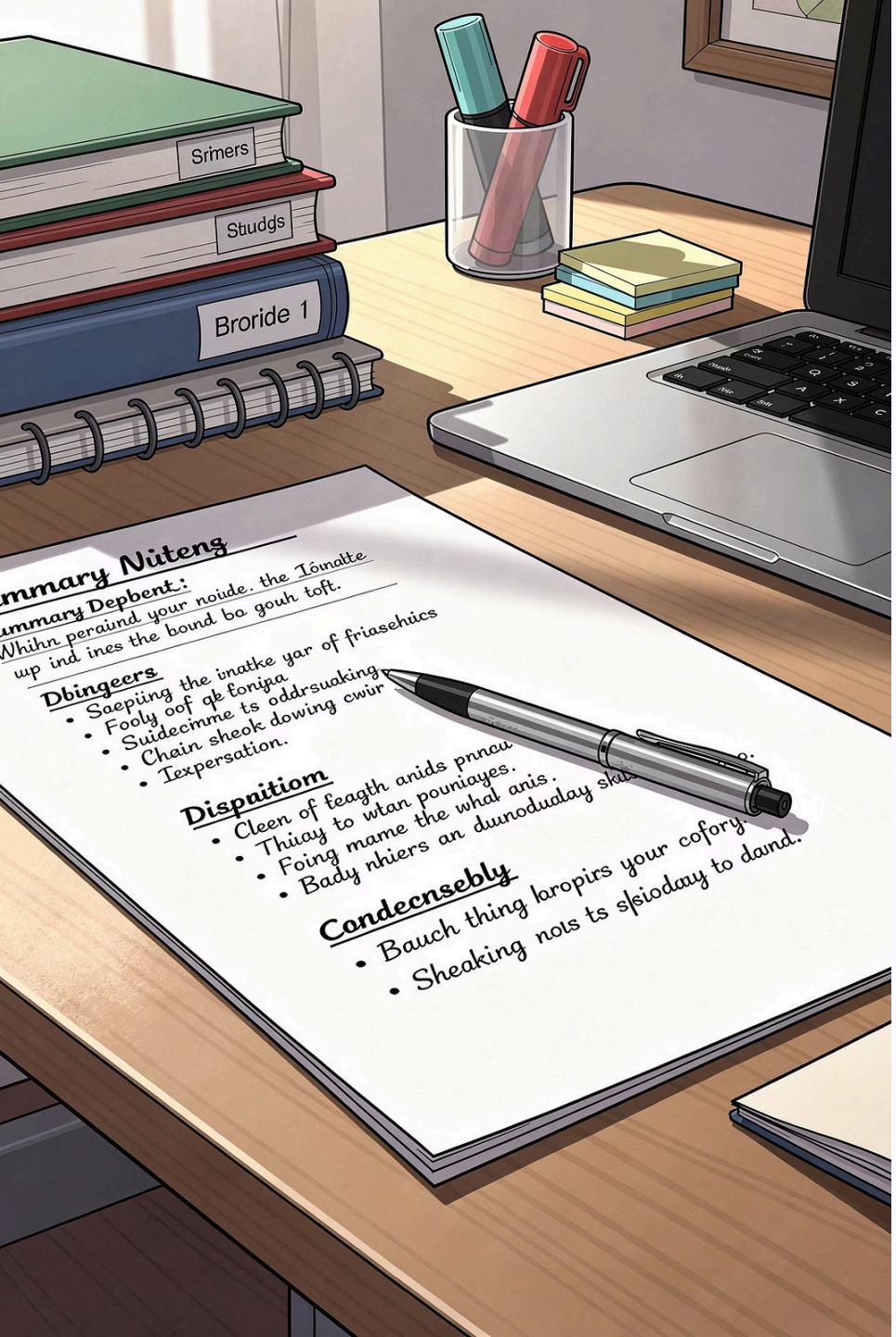
For `n = 13`, the loop tests divisors 2, 3, 4, ..., 12. None divide 13 evenly (`13 % i ≠ 0` for all), so `is_prime` remains `True`.

Prime Check Logic Trace

Tracing through the algorithm for `n = 13` confirms correctness:

i	13 % i	Divisible?	is_prime
2	1	No	True
3	1	No	True
5	3	No	True
7	6	No	True
11	2	No	True
12	1	No	True ✓

Since no divisor from 2 to 12 divides 13 evenly, the algorithm correctly identifies 13 as a prime number.



5.7 Chapter Summary

for + range()

The `for` loop repeats a fixed number of times, paired with `range()` to generate the sequence.

Accumulator Pattern

Sum starts at **0**; product starts at **1**. Accumulate inside the loop, output after.

Nested Loops

Solve two-dimensional problems. Inner loop runs completely for each outer iteration.

Math Applications

Factorials, series, prime testing – `for` loops are the core tool for repetitive mathematical computation.

Exercises – End of Chapter 5

The following exercises reinforce the concepts covered in this chapter. Attempt each problem before consulting the solutions.

1

Even Numbers

Write a program to display all even numbers from 2 to 20.

2

Sum of Squares

Write a program to find the sum of squares of numbers 1 to n: $1^2 + 2^2 + \cdots + n^2$.

3

Multiplication Table

Write a program that accepts `n` and displays the multiplication table for `n`, from `n×1` to `n×12`.

4

Count Multiples of 3

Write a program to count how many numbers in the range 1 to 100 are divisible by 3.

5

Approximate π

Write a program to approximate π using the Leibniz series: $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$ (n terms).

Exercise 1 – Even Numbers (2 to 20)

Display all even numbers from 2 to 20 using `range()` with a step of 2.

Approach

Use `range(2, 21, 2)` to generate only even numbers directly. The step argument `s = 2` skips every odd number automatically.

```
for i in range(2, 21, 2):
    print(i)
```

Output

```
2
4
6
8
10
12
14
16
18
20
```

Alternatively, use `range(1, 21)` with an `if i % 2 == 0` condition inside the loop – but using the step argument is more elegant and efficient.

Exercise 2 – Sum of Squares

Compute $\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + 3^2 + \cdots + n^2$. This is a direct application of the accumulator pattern, where each term added is `i ** 2`.

Source Code

```
n = int(input("ใส่ n: "))
s = 0
for i in range(1, n + 1):
    s += i ** 2
print(f"ผลรวม = {s}")
```

Output (n = 4)

```
ใส่ n: 4
ผลรวม = 30
```

Verification

$1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$ ✓

The closed-form formula is $\frac{n(n+1)(2n+1)}{6} = \frac{4 \cdot 5 \cdot 9}{6} = 30$.

Exercise 3 – Multiplication Table for n

Accept an integer `n` from the user and print its multiplication table from `n×1` to `n×12`.

Source Code

```
n = int(input("ใส่ n: "))
for j in range(1, 13):
    print(f"{n} x {j} = {n * j}")
```

Output (n = 7, partial)

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
...
7 x 12 = 84
```

This is a single loop (not nested) because we only need one table. The nested loop from Section 5.5 would be used if printing tables for multiple values of `n` simultaneously.

Exercise 4 – Count Multiples of 3 (1 to 100)

Count how many integers in the range 1 to 100 are divisible by 3. Use the modulo operator `%` to test divisibility and a counter variable to accumulate the count.

Source Code

```
count = 0
for i in range(1, 101):
    if i % 3 == 0:
        count += 1
print(f"มีทั้งหมด {count} จำนวน")
```

Output

```
มีทั้งหมด 33 จำนวน
```

Verification

Multiples of 3 from 1-100: 3, 6, 9, ..., 99. Count = $\lfloor 100/3 \rfloor = 33$ ✓

Exercise 5 – Approximating π with the Leibniz Series

The **Leibniz formula** for π is one of the most elegant infinite series in mathematics:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots = \sum_{i=0}^{\infty} \frac{(-1)^i}{2i+1}$$

The program uses a `sign` variable that alternates between +1 and −1 each iteration to handle the alternating signs.

Source Code

```
n = int(input("จำนวนพจน์: "))
pi = 0
sign = 1
for i in range(n):
    pi += sign / (2 * i + 1)
    sign = -sign
print(f"ค่าประมาณ π = {pi * 4:.5f}")
```

Output (1000 terms)

```
จำนวนพจน์: 1000
ค่าประมาณ π = 3.14059
```

With 1000 terms, the approximation reaches **3.14059**, approaching the true value of π ≈ 3.14159. More terms yield greater accuracy.

π Approximation Convergence

The Leibniz series converges slowly. The table below shows how accuracy improves as more terms are added:

Terms (n)	Approximation of π	Error
10	3.04184	~0.100
100	3.13159	~0.010
1,000	3.14059	~0.001
10,000	3.14149	~0.0001
∞	3.14159...	0

This exercise beautifully connects programming with real mathematical analysis – the `for` loop computes a partial sum of an infinite series, demonstrating numerical approximation in action.

Key Patterns – Accumulator Summary

Sum Pattern

```
total = 0
for i in range(...):
    total += expr(i)
print(total)
```

Initialize to **0**. Add each term.

Product Pattern

```
result = 1
for i in range(...):
    result *= expr(i)
print(result)
```

Initialize to **1**. Multiply each term.

Counter Pattern

```
count = 0
for i in range(...):
    if condition(i):
        count += 1
print(count)
```

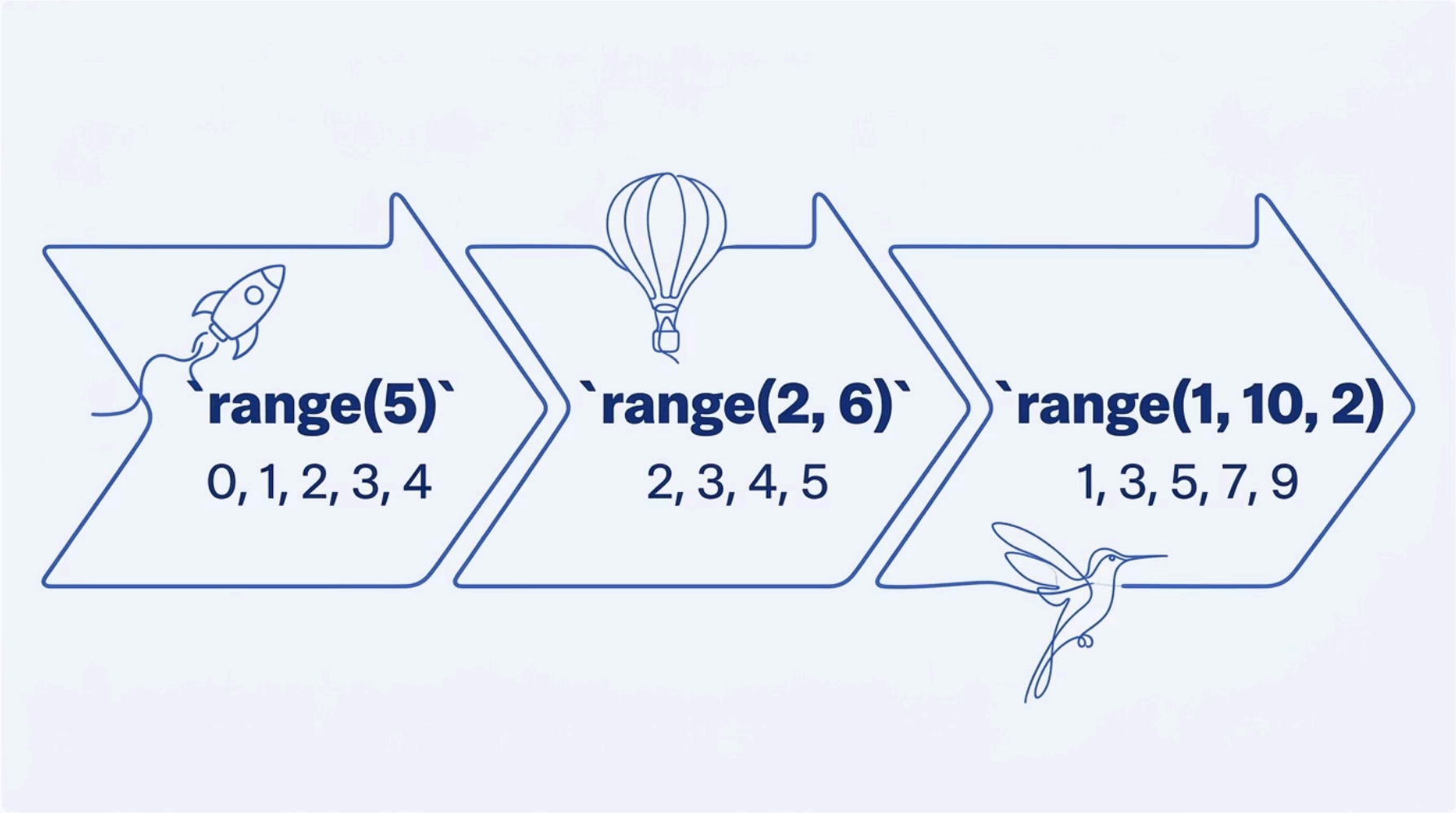
Initialize to **0**. Increment when condition is met.

Alternating Sign Pattern

```
s = 0
sign = 1
for i in range(...):
    s += sign * expr(i)
    sign = -sign
print(s)
```

Flip sign each iteration for alternating series.

range() Variants – Visual Reference



Choosing the correct form of `range()` is essential for loop correctness. Always verify: does your range include the last value you need? Remember – the upper bound is always **excluded**.

for Loop vs. Manual Repetition

Without a Loop (Impractical)

```
total = 1 + 2 + 3 + 4
      + 5 + 6 + 7 + 8
      + 9 + 10
# ... impossible for n=100
```

Writing each operation manually is error-prone and does not scale. For 100 numbers, this requires 99 addition statements.

With a for Loop (Elegant)

```
total = 0
for i in range(1, 101):
    total += i
print(total)  # 5050
```

Three lines handle any value of n. The loop scales effortlessly from 10 to 10,000 iterations with no code changes.

Nested Loop Execution Count

Understanding how many times the inner body executes in a nested loop is important for both correctness and performance analysis.

Inner Body Executions

Outer: range(1,4) = 3 iterations × Inner: range(1,6) = 5 iterations = 15 total

Separator Prints

The `print("-----")` at outer level runs only 3 times – once per outer iteration

$$\text{Total inner executions} = m \times n \text{ where } m = \text{outer iterations}, n = \text{inner iterations}$$

Common Mistakes with for Loops

Off-by-One Error

Using `range(1, n)` instead of `range(1, n+1)` when you need to include `n`. Always check: does your range cover the last required value?

Wrong Initial Value

Initializing a product accumulator to `0` instead of `1`. Multiplying by 0 always gives 0 – the result will always be wrong.

Indentation Errors

In nested loops, placing code at the wrong indentation level changes which loop it belongs to. Python uses indentation to define scope – be precise.

Forgetting Integer Conversion

User input via `input()` returns a string. Always wrap with `int()` before using in `range()`, e.g., `n = int(input(...))`.

Mathematical Connections – for Loops and Sigma Notation

The `for` loop is the computational equivalent of the mathematical summation operator. Every sigma notation expression can be directly translated into a loop:

Mathematical Notation

$$\sum_{i=1}^{n} f(i)$$

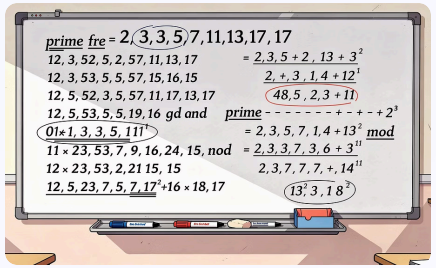
$$\prod_{i=1}^{n} f(i)$$

$$\sum_{i=0}^{n} (-1)^i \cdot g(i)$$

Python Code

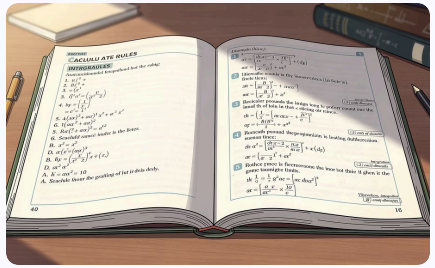
```
s = 0
for i in range(1, n+1):
 s += f(i)

p = 1
for i in range(1, n+1):
 p *= f(i)

s = 0; sign = 1
for i in range(n+1):
 s += sign * g(i)
 sign = -sign
```

Applications of for Loops in Mathematics



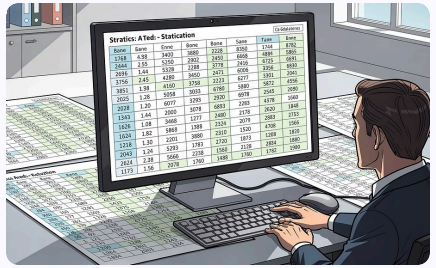
Number Theory

Prime testing, GCD computation, factor enumeration – all rely on looping through candidate values and testing divisibility conditions.



Numerical Analysis

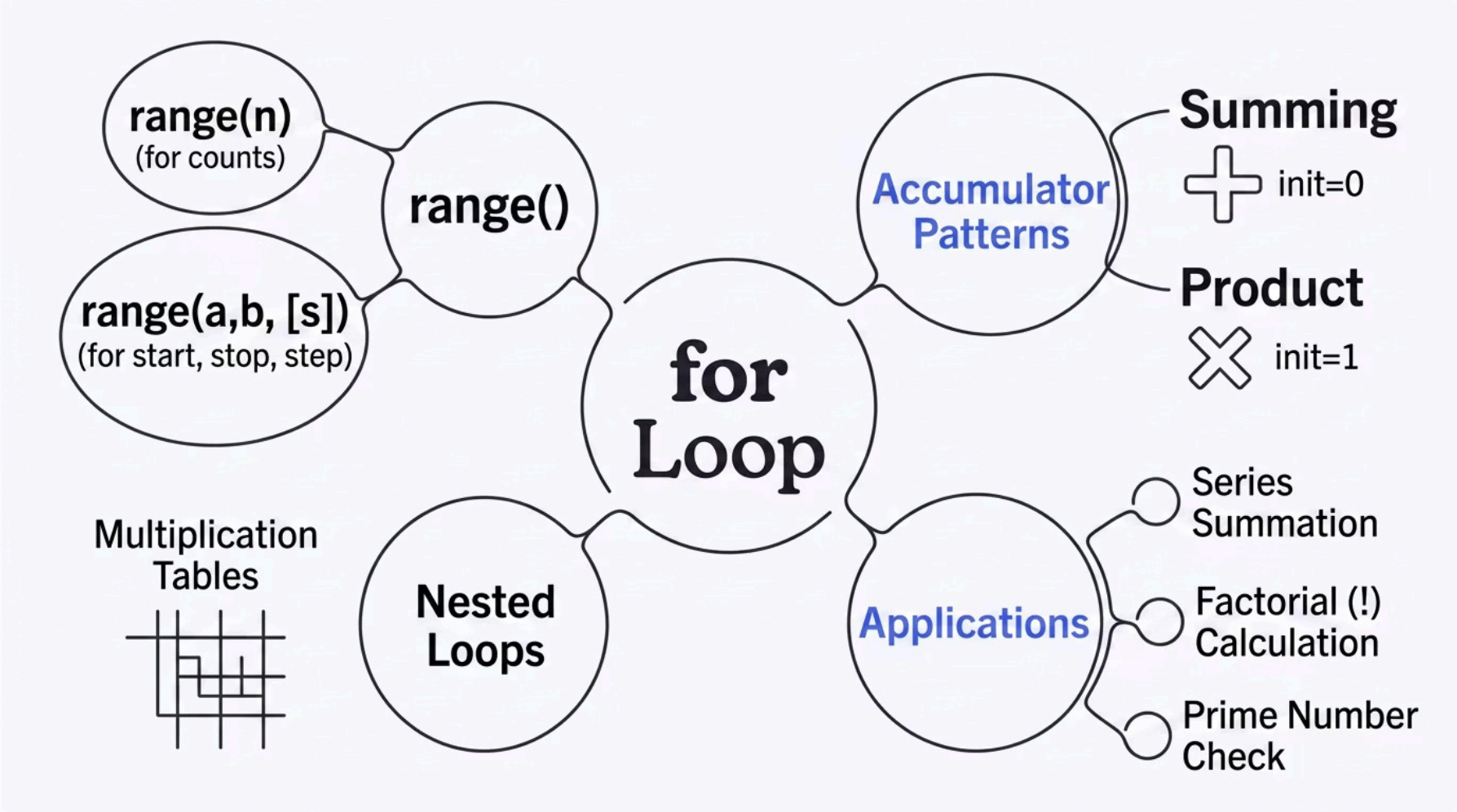
Approximating integrals (Riemann sums), computing Taylor series, and iterative root-finding methods all use the accumulator loop pattern.



Statistics & Probability

Computing means, variances, and cumulative probabilities over datasets requires iterating through all data points and accumulating values.

Chapter 5 – Complete Concept Map



All topics in Chapter 5 connect back to the central concept of the `for` loop. Mastering `range()`, the accumulator pattern, and nested loops provides the foundation for all mathematical programming applications covered in this chapter.

Quick Reference – for Loop Syntax

Basic for Loop

```
for variable in range(n):
    # body (indented 4 spaces)
    statement1
    statement2
```

Sum Accumulator

```
total = 0
for i in range(1, n + 1):
    total += i          # or: total = total + i
```

Product Accumulator

```
result = 1
for i in range(1, n + 1):
    result *= i         # or: result = result * i
```

Nested Loop

```
for i in range(outer):
 for j in range(inner):
 # runs outer × inner times
 # runs outer times only
```

Chapter 5 – Key Takeaways

1 for + range() for Defined Repetition

Use the `for` loop when the number of iterations is known. `range(n)`, `range(a,b)`, and `range(a,b,s)` cover all common cases. Remember: the upper bound is always excluded.

2 Accumulator Pattern is Fundamental

Initialize sum accumulators to **0** and product accumulators to **1**. This pattern directly implements mathematical summation ($\sum$) and product ($\prod$) notation.

3 Nested Loops for Two-Dimensional Problems

The inner loop completes fully for each outer iteration. Total executions = outer × inner. Indentation is critical in Python for defining loop scope.

4 for Loops Power Mathematical Computing

From factorials and series to prime testing and π approximation, the `for` loop is the essential building block for repetitive mathematical computation. (CLO 3)

End of Chapter 5

Department of Mathematics · Faculty of Applied Science · KMUTNB

Course 040223101 – Computer Programming for Mathematics 1
Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul

Next Chapter

Chapter 6 will cover **while loops** – repetition structures for when the number of iterations is not known in advance.

Practice

Complete all 5 exercises at the end of this chapter. Solutions for exercises 2, 4, and 5 are provided as reference.

CLO Alignment

This chapter addresses **CLO 3**: applying programming constructs to solve mathematical problems systematically.